

Terraform Security Best Practices

A practical guide to securing Infrastructure as Code

Covers secrets management, state file security, identity and access, code and module hygiene, CI/CD pipeline security, and ongoing drift and version hygiene — with current guidance on Terraform 1.10/1.11 ephemeral values, S3 native state locking, and dynamic provider credentials.

August 2026

Contents

- 01 Introduction
- 02 Secrets and Sensitive Data Management
- 03 State File Security
- 04 Identity and Access Control
- 05 Code and Module Security
- 06 Network and Resource Hardening
- 07 CI/CD Pipeline Security
- 08 Drift Detection and Ongoing Hygiene
- 09 Quick-Reference Checklist
- 10 Sources and Further Reading

01 Introduction

Terraform configurations describe the exact shape of your cloud infrastructure, and Terraform state files often hold a working copy of every attribute of every resource you manage — including plenty that are sensitive. A leaked state file, an over-permissioned CI credential, or an unreviewed module can hand an attacker the same blast radius as a compromised admin console. Treat your Terraform code, state, and pipeline with the same rigor you'd apply to production credentials, because in practice they *are* production credentials.

This guide is organized around the points in a typical Terraform workflow where security actually breaks down in practice: the secrets referenced in configuration, the state file that records the result, the identity Terraform runs as, the code and modules being applied, the infrastructure that code provisions, and the pipeline that ties it all together.

02 Secrets and Sensitive Data Management

Never hardcode secrets in configuration

Database passwords, API keys, and tokens should never appear as literal values in .tf files or in .tfvars files that get committed to version control. Add *.tfvars, *.auto.tfvars, and *.tfstate* to .gitignore as a baseline, and run a pre-commit secret scanner such as gitleaks or detect-secrets so an accidental commit is caught before it ever reaches a shared branch.

```
# .gitignore
*.tfvars
*.tfvars.json
*.tfstate
*.tfstate.backup
.terraform/
```

Pull secrets from a secrets manager, not from variables

Reference secrets from a dedicated secrets manager at plan/apply time instead of passing them in as plain variables:

```
data "aws_secretsmanager_secret_version" "db" {
  secret_id = "prod/app/db-password"
}

resource "aws_db_instance" "main" {
  # ...
  password = data.aws_secretsmanager_secret_version.db.secret_string
}
```

The equivalent pattern applies to HashiCorp Vault (vault_generic_secret), Azure Key Vault (azurerm_key_vault_secret), and Google Secret Manager (google_secret_manager_secret_version).

sensitive = true redacts output — it does not encrypt

Marking a variable or output as sensitive stops Terraform from printing its value in plan/apply output and in the CLI, which is worth doing for every credential-shaped value. But it is a display-layer protection only: the real value is still written to the state file in plain text. Don't treat sensitive = true as encryption — see Section 3 for how to actually protect state.

```
variable "db_password" {  
  type      = string  
  sensitive = true  
}
```

Keep secrets out of state entirely: ephemeral values and write-only arguments

Terraform 1.10 introduced **ephemeral values** — ephemeral resources, variables, and outputs whose values are used during a plan or apply but are never written to the state file or plan file at all. Terraform 1.11 extended this to managed resources through **write-only arguments**: a resource attribute marked write-only (for example, a database master password) is sent to the provider but never persisted to state, and Terraform re-supplies it from your configuration or an ephemeral source on every apply instead of reading it back from state.

```
ephemeral "random_password" "db" {  
  length = 20  
  special = true  
}  
  
resource "aws_db_instance" "main" {  
  # write-only argument: never stored in state  
  password_wo      = ephemeral.random_password.db.result  
  password_wo_version = 1  
}
```

Note: This is genuinely new ground for Terraform's security model as of the 1.10/1.11 releases, not a rename of an older feature — where a provider and resource support it, prefer ephemeral values and write-only arguments over sensitive = true for any credential that doesn't need to persist for drift detection.

03 State File Security

The state file is, in effect, a snapshot of your infrastructure's configuration and secrets. Anyone who can read it can typically reconstruct sensitive attributes of every resource it tracks. Anyone who can write to it can corrupt or hijack your infrastructure on the next apply.

Use a remote backend, never local state

Local state (terraform.tfstate on a laptop) has no locking, no access control, and no audit trail, and it gets committed to git by accident more often than teams like to admit. Use a remote backend — S3, Azure Storage, Google Cloud Storage, or HCP Terraform/Terraform Enterprise — for any state shared by more than one person or pipeline.

Enable locking

Locking prevents two concurrent applies from corrupting the same state. As of Terraform 1.10, the S3 backend supports native locking through S3 conditional writes, so a separate DynamoDB lock table is no longer required on new setups:

```
terraform {
  backend "s3" {
    bucket      = "my-org-terraform-state"
    key         = "prod/network/terraform.tfstate"
    region      = "us-east-1"
    encrypt     = true
    use_lockfile = true    # Terraform >= 1.10: native S3 locking, no DynamoDB
  }
}
```

Note: If you're on Terraform < 1.10 or maintaining an older setup, the dynamodb_table argument still works and is fully supported — there's no need to migrate an existing, working DynamoDB-locked backend purely for its own sake.

Encrypt state at rest

Terraform itself has no built-in, client-side state encryption — it relies entirely on the backend's own encryption. In practice that means turning on server-side encryption on the storage backend (S3 bucket default encryption with a customer-managed KMS key, Azure Storage encryption, or GCS default encryption), plus bucket versioning so a bad apply or accidental delete can be recovered from a prior version:

```

resource "aws_s3_bucket" "state" {
  bucket = "my-org-terraform-state"
}

resource "aws_s3_bucket_versioning" "state" {
  bucket = aws_s3_bucket.state.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state" {
  bucket = aws_s3_bucket.state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
      kms_master_key_id = aws_kms_key.state.arn
    }
  }
}

resource "aws_s3_bucket_public_access_block" "state" {
  bucket = aws_s3_bucket.state.id
  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}

```

If you specifically need client-side envelope encryption of the state file itself (so even the storage backend never sees plaintext), that's a differentiator of OpenTofu rather than HashiCorp Terraform: OpenTofu has shipped native state encryption (with AWS KMS, GCP KMS, Vault, and PBKDF2 key providers) since its 1.7 release.

Lock down who can read and write state

- Give the state bucket its own IAM policy, separate from application data buckets, and grant access only to the roles/users that actually run Terraform.
- Block public access on the bucket and disable ACLs.
- Never commit terraform.tfstate or terraform.tfstate.backup to version control, even to a private repository.
- Use separate state files (or workspaces) per environment, so a compromised or careless dev credential can't read or write prod state.

04 Identity and Access Control

Grant Terraform's own credentials least privilege

The IAM role or service principal Terraform runs as should be scoped to exactly the resource types and actions the configuration actually manages, not broad administrator access. Split this further by

environment and, where practical, by component, so a compromised credential for one workspace can't touch every workspace.

Prefer short-lived, dynamic credentials over static keys

Long-lived static access keys sitting in a CI secret store are one of the highest-value targets in a typical pipeline. HCP Terraform and Terraform Enterprise support **dynamic provider credentials**: for each run, an OIDC-compliant identity token is issued, exchanged with the cloud provider for short-lived credentials, and discarded when the run finishes. This is supported today for AWS, Azure, GCP, Kubernetes, and Vault.

```
# HCP Terraform workspace environment variables (no static keys)
TFC_AWS_PROVIDER_AUTH      = true
TFC_AWS_RUN_ROLE_ARN       = "arn:aws:iam::123456789012:role/terraform-ci"
```

Running outside HCP Terraform (plain GitHub Actions, GitLab CI, etc.)? The same idea applies directly: configure the cloud provider to trust your CI platform's OIDC issuer and assume a role for the duration of the job, instead of storing an access key as a CI secret.

Scope down further with assume-role and session policies

- Use a dedicated assume-role (with an external ID for cross-account access) rather than a general-purpose account credential.
- Attach a session policy to further restrict what a given run can touch, on top of the role's own permissions.
- Rotate any credential that can't be made dynamic, and alert on its use outside expected pipeline windows.

05 Code and Module Security

Pin provider and module versions

An unpinned provider or a module sourced from a floating branch can change behavior under you between runs, including security-relevant defaults. Pin providers with a version constraint and commit the resulting `.terraform.lock.hcl`; pin modules to a tag or commit SHA, not a branch name.

```

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.60"
    }
  }
}

module "vpc" {
  source = "git::https://example.com/modules/vpc.git?ref=v2.3.1"
  # not: ref=main
}

```

Vet third-party modules before use

- Prefer modules from your organization's private registry or the official Terraform Registry's verified publishers.
- Read the source of any community module before pointing production at it — a module can create IAM policies, security groups, and secrets just as easily as your own code.
- Re-review a module on every version bump, not just the first time it's adopted.

Scan code before it's applied

Run static analysis in CI, before terraform plan is even shown to a reviewer:

- **terraform validate** and **terraform fmt -check** for basic correctness and consistency.
- **Trivy** for infrastructure-as-code misconfiguration scanning — it absorbed tfsec's rule set and checks after Aqua Security merged the two projects, so new setups should target Trivy directly rather than installing tfsec.
- **Checkov** or **Terrascan** as complementary scanners with broader compliance-framework coverage (CIS, SOC 2, HIPAA, PCI-DSS mappings).

```

# CI step (example)
trivy config --severity HIGH,CRITICAL ./terraform
checkov -d ./terraform

```

Enforce guardrails with policy as code

Static scanners catch known-bad patterns; policy as code lets you encode your own organization's rules (allowed regions, required tags, instance size ceilings, mandatory encryption) and block a plan that violates them before apply. Sentinel is built into HCP Terraform and Terraform Enterprise; OPA with Conftest is the open-source equivalent and runs in any CI system against a terraform plan -out and its JSON representation.

06 Network and Resource Hardening

Terraform security isn't only about the tool — it's also about what the code tells your cloud provider to build. A perfectly secured pipeline can still ship a wide-open security group.

- Default security groups and NSGs to deny-all, and open only the specific ports and source ranges a workload needs — never 0.0.0.0/0 on a database or management port.
- Put compute and data resources in private subnets; require a load balancer, bastion, or SSM/session-manager style access for anything that needs to reach them.
- Turn on encryption at rest for every resource that offers it as a flag — EBS volumes, RDS instances, S3 buckets, managed queues — and require TLS for anything in transit.
- Provision your audit trail as code alongside the workload: CloudTrail (or your provider's equivalent), VPC/flow logs, and config/compliance rules so drift and unauthorized changes are visible.

07 CI/CD Pipeline Security

- Run plan and apply from a controlled pipeline for any shared environment — not from an engineer's laptop with personal credentials.
- Require a human review of the plan output on a pull request before apply runs against a protected environment, and require a second approver for production.
- Use OIDC-based dynamic credentials in the pipeline itself (Section 4) so the CI platform never stores a long-lived cloud credential.
- Keep any secret the pipeline still needs (registry tokens, webhook secrets) in the CI platform's native secret store, scoped to the fewest jobs possible — never in the repository.
- Give each environment its own state, workspace, and pipeline stage, so a mistake applied to dev has no path to prod.

08 Drift Detection and Ongoing Hygiene

- Schedule a recurring terraform plan (or your platform's drift-detection feature) against every environment to catch changes made outside Terraform before they become a surprise.
- Keep Terraform core and provider versions current — security fixes ship in point releases, and the gap between your pinned version and current only grows more painful to close over time.
- Rotate the credentials Terraform itself runs as on a regular cadence, even when they're not static long-lived keys.
- Tag or label every resource with an owner and environment so a security review can trace any resource back to the configuration and team responsible for it.

09 Quick-Reference Checklist

A condensed list to run through before your next apply against a shared or production environment.

Area	Check
Secrets	■ No plaintext secrets in .tf or committed .tfvars; secrets sourced from a secrets manager
Secrets	■ Sensitive variables/outputs marked sensitive = true
Secrets	■ Ephemeral values / write-only arguments used where the provider and resource support them
Secrets	■ Pre-commit secret scanning enabled (gitleaks / detect-secrets)
State	■ Remote backend in use; no local state for shared environments
State	■ Locking enabled (S3 native use_lockfile, or DynamoDB on older setups)
State	■ State bucket/container encrypted at rest, versioned, and blocked from public access
State	■ Separate state per environment
Identity	■ Terraform's own credentials scoped to least privilege
Identity	■ Dynamic/OIDC credentials used instead of static long-lived keys wherever possible
Code	■ Provider versions pinned; lock file committed
Code	■ Modules pinned to a tag or commit SHA, not a branch
Code	■ Trivy/Checkov (or equivalent) run in CI on every plan
Code	■ Policy-as-code guardrails (Sentinel or OPA/Conftest) enforced before apply
Pipeline	■ Plan reviewed by a human before apply on protected environments
Pipeline	■ Apply runs from CI/CD, not a personal laptop, for shared environments
Hygiene	■ Scheduled drift detection in place
Hygiene	■ Terraform core and provider versions kept current

10 Sources and Further Reading

This guide draws on HashiCorp's own documentation and release notes, plus independent coverage of tooling changes current as of mid-2026:

- HashiCorp Developer — Dynamic Provider Credentials in HCP Terraform: developer.hashicorp.com/terraform/cloud-docs/dynamic-provider-credentials
- HashiCorp Blog — Terraform 1.10 improves handling secrets in state with ephemeral values: hashicorp.com/blog/terraform-1-10-improves-handling-secrets-in-state-with-ephemeral-values
- HashiCorp Blog — Terraform 1.11 brings ephemeral values to managed resources with write-only arguments: hashicorp.com/blog/terraform-1-11-ephemeral-values-managed-resources-write-only-arguments
- HashiCorp Blog — Simplifying Terraform dynamic credentials on AWS with native OIDC integration: hashicorp.com/blog/simplifying-terraform-dynamic-credentials-on-aws-with-native-oidc-integration
- HashiCorp Developer — Manage sensitive data in your configuration: developer.hashicorp.com/terraform/language/manage-sensitive-data
- freeCodeCamp — How to Migrate to S3 Native State Locking in Terraform: freecodecamp.org/news/how-to-migrate-to-s3-native-state-locking-in-terraform
- Spacelift — How to Encrypt Terraform State Files (state has no native Terraform encryption; OpenTofu native encryption since v1.7): spacelift.io/blog/terraform-state-encryption
- Spacelift — What Is tfsec? Installation, Configuration & Migration to Trivy: spacelift.io/blog/what-is-tfsec
- env0 — Terraform Governance Tools Compared for 2026 (OPA, Sentinel, Checkov, Trivy): env0.com/insights/terraform-governance-tools-compared-opa-sentinel-checkov-tfsec-and-when-to-use-each

This document reflects publicly documented Terraform/HCP Terraform behavior as of August 2026. Feature availability can vary by Terraform version and by whether you use open-source Terraform, HCP Terraform, or Terraform Enterprise — verify against your own version before relying on any specific flag or block shown here.